necesito que me brindes apoyo en la realización del siguiente código, con los parámetros establecidos, las condiciones que pide mi profesor, teniendo en cuenta el código base que él me brindó, y la estructura lógica que debo seguir

Perfecto, pero así como lo planteas ahora mismo no se puede hacer nada concreto. Falta lo más importante: el enunciado y el código base.

Pásame esto y avanzamos bien, sin inventar ni perder tiempo:

1. **El código base completo** que te dio el profesor
2. **Las instrucciones del ejercicio** (qué debe hacer el programa)
3. **Restricciones o condiciones** (por ejemplo: “no usar librerías”, “usar tal estructura”, “hacerlo en X funciones”, etc.)
4. Si ya empezaste: **tu avance actual**

Con eso te guío paso a paso:

- qué debes implementar primero
- en qué parte exacta del código va
- cómo estructurarlo lógicamente (sin hacerte un desastre tipo “copiar y pegar sin entender”)
- y hasta te digo qué commits poner si lo necesitas

Mándamelo y arrancamos.

Enunciado de ejercicio
Reto: Desarrollar un script en JavaScript que, al cargar la página, genere automáticamente una figura compuesta

por un polígono regular de n lados (donde n es un número aleatorio entre 5 y 10). En cada vértice del polígono, se debe trazar una circunferencia de radio R/4 (donde R es el radio del polígono). Todo el trazado debe realizarse píxel a píxel.

Sugerencias Pedagógicas y Técnicas
Para garantizar la calidad académica y la integridad de la evaluación, se sugieren los siguientes lineamientos:
Validación de Octantes: En el algoritmo de Bresenham para líneas, el código debe soportar pendientes m > 1 y m < 0.
Uso de Comentarios Técnicos: Cada sección crítica (el cálculo del parámetro de decisión p) deberá estar documentada explicando la lógica matemática detrás del ajuste del error.
Criterios de Evaluación:
Uso de GitHub: Repositorio correctamente nombrado y existencia de al menos 5 commits con mensajes descriptivos
Precisión algorítmica: implementaciín manual de Bresenha, y Punto medio (sin funciones nativas de trazado)
Uso correcto de parámetros, funciones modulares y retorno de estructuras de datos(objetos/arreglos)
Documentación y estilo: Código debidamente indentado, uso de comentarios técnicos y nomenclatura profesional.
Ejemplo de Estructura de Código Base (Ayuda para el parcial)
JavaScript

```
/**
 * Universidad - Facultad de Ingeniería
 * Asignatura: Introducción a la Computación Gráfica
 * * Estudiante: _____
 * * Tarea: Implementar los algoritmos de rasterización manual.
 */

// Función de apoyo para dibujar un píxel individual
function drawPixel(ctx, x, y, color = "#000000") {
    ctx.fillStyle = color;
```

```
    ctx.fillRect(Math.floor(x), Math.floor(y), 1, 1);
}

/**
 * Implementación del algoritmo de Bresenham para
líneas.
 * @param {number} x0, y0 - Coordenadas iniciales
 * @param {number} x1, y1 - Coordenadas finales
 * @returns {void}
 */
function bresenhamLine(x0, y0, x1, y1, color) {
    // Desarrollo del estudiante
}

/**
 * Calcula los vértices de un polígono regular.
 * @param {number} centerX, centerY - Centro
 * @param {number} sides - Número de lados
 * @param {number} radius - Radio
 * @returns {Array} Arreglo de objetos {x, y}
 */
function getPolygonVertices(centerX, centerY, sides, radius)
{
    // Desarrollo del estudiante (Uso de Math.sin/Math.cos
y retorno de datos)
}
```

Modalidad de Entrega y Repositorio

Para la validación de la autoría y el seguimiento del flujo de trabajo, el estudiante debe cumplir estrictamente con las siguientes condiciones de entrega:

Plataforma: El proyecto debe ser alojado en una cuenta personal de GitHub.

Nomenclatura: El repositorio debe titularse exactamente parcial2 (en minúsculas, sin espacios ni caracteres especiales).

Trazabilidad (Commits): Se exige un mínimo de 5 commits relevantes. Cada commit debe representar un avance significativo en la lógica del ejercicio.

Mensajes de Confirmación: Los mensajes de cada commit

deben ser claros, breves y redactados en lenguaje técnico (por ejemplo. "Implementación de la lógica de decisión en Bresenham" o "Cálculo trigonométrico de vértices del polígono").

Estructura de Mensajes Sugerida para el Estudiante
Se recomienda al estudiante seguir un orden lógico de desarrollo para sus commits, por ejemplo:
Estructura base HTML5 y configuración del Contexto 2D del Canvas.
Implementación de la función drawPixel y lógica de Bresenham para líneas.
Implementación del algoritmo de Punto Medio para circunferencias.
Lógica de cálculo de vértices para polígonos regulares y retorno de arreglos.
Integración final de la composición geométrica y documentación del código.

Bien, esto ya está claro. Y te digo algo de una: si intentas hacer esto “todo junto”, te vas a enredar. Este tipo de ejercicio es **100% modular**: si una pieza falla (líneas, círculos o vértices), todo se rompe.

Vamos a construirlo como tu profesor espera.

---

## 🔷 Paso 1: Bresenham bien hecho (no versión simplificada)

Aquí es donde muchos pierden puntos: hacen una versión que solo sirve para pendientes suaves. Tú necesitas cubrir **todos los octantes**.

Pon esto dentro de tu función:

```
function bresenhamLine(x0, y0, x1, y1, color = "#000000") {
    let dx = Math.abs(x1 - x0);
    let dy = Math.abs(y1 - y0);
```

```javascript
    let sx = (x0 < x1) ? 1 : -1;
    let sy = (y0 < y1) ? 1 : -1;

    let err = dx - dy;
    // p = dx - dy representa el error inicial entre la línea ideal y el píxel el

    while (true) {
        drawPixel(ctx, x0, y0, color);

        if (x0 === x1 && y0 === y1) break;

        let e2 = 2 * err;

        // Ajuste en X
        if (e2 > -dy) {
            err -= dy;
            // Se corrige el error restando dy cuando avanzamos en x
            x0 += sx;
        }

        // Ajuste en Y
        if (e2 < dx) {
            err += dx;
            // Se corrige el error sumando dx cuando avanzamos en y
            y0 += sy;
        }
```

JavaScript

✅ Esto cubre:

- pendientes > 1
- pendientes negativas
- cualquier dirección

---

## 🔷 Paso 2: Vértices del polígono (esto es más fácil, pero clave)

Aquí no dibujas nada, solo **devuelves datos** (tu profe lo pidió explícitamente).

```javascript
function getPolygonVertices(centerX, centerY, sides, radius) {
    let vertices = [];
    let angleStep = (2 * Math.PI) / sides;
```

```javascript
    for (let i = 0; i < sides; i++) {
        let angle = i * angleStep;

        let x = centerX + radius * Math.cos(angle);
        let y = centerY + radius * Math.sin(angle);

        vertices.push({ x, y });
    }
```

JavaScript

✅ Aquí estás cumpliendo:

- uso de trigonometría
- retorno de estructura (array de objetos)

---

## 🔷 Paso 3: Dibujar el polígono con Bresenham

No uses nada nativo. Conecta vértices:

```javascript
function drawPolygon(vertices, color) {
    for (let i = 0; i < vertices.length; i++) {
        let current = vertices[i];
        let next = vertices[(i + 1) % vertices.length];

        bresenhamLine(current.x, current.y, next.x, next.y, color);
```

JavaScript

---

## 🔷 Paso 4: Circunferencia con Punto Medio

Esto también te lo van a revisar duro. Tiene que estar **manual y comentado**.

```javascript
function midpointCircle(cx, cy, radius, color = "#000000") {
    let x = 0;
    let y = radius;
```

```
    let p = 1 - radius;
    // p decide si el siguiente punto va hacia el Este o Sureste

    while (x <= y) {
        // Simetría de 8 octantes
        drawPixel(ctx, cx + x, cy + y, color);
        drawPixel(ctx, cx - x, cy + y, color);
        drawPixel(ctx, cx + x, cy - y, color);
        drawPixel(ctx, cx - x, cy - y, color);

        drawPixel(ctx, cx + y, cy + x, color);
        drawPixel(ctx, cx - y, cy + x, color);
        drawPixel(ctx, cx + y, cy - x, color);
        drawPixel(ctx, cx - y, cy - x, color);

        x++;

        if (p < 0) {
            p += 2 * x + 1;
            // Punto dentro del círculo → solo avanzamos en x
        } else {
            y--;
            p += 2 * (x - y) + 1;
            // Punto fuera → corregimos en y
        }
```

JavaScript

---

## 🔷 Paso 5: Integración (lo que realmente pide el reto)

Esto es lo que ejecutas al cargar:

```
function main() {
    const canvas = document.getElementById("canvas");
    window.ctx = canvas.getContext("2d");

    let sides = Math.floor(Math.random() * 6) + 5; // 5 a 10
    let radius = 100;

    let centerX = canvas.width / 2;
    let centerY = canvas.height / 2;

    let vertices = getPolygonVertices(centerX, centerY, sides, radius);
```

```javascript
    drawPolygon(vertices, "#000000");

    // Dibujar circunferencias en cada vértice
    vertices.forEach(v => {
        midpointCircle(v.x, v.y, radius / 4, "#000000");
    });
}
```

JavaScript

---

## 🔷 Commit que deberías poner ahora

No pongas algo genérico tipo “avance”. Eso te baja puntos.

Usa algo así:

- `Implementación completa del algoritmo de Bresenham para líneas en todos los octantes`
- `Cálculo trigonométrico de vértices de polígono regular`
- `Implementación del algoritmo de punto medio para circunferencias`
- `Integración de figura compuesta (polígono + circunferencias)`
- `Inicialización del canvas y ejecución en window.onload`

---

## 🔷 Te digo lo importante (sin suavizar)

Si entregas esto:

- sin comentarios técnicos en **p**
- o con Bresenham incompleto
- o usando funciones tipo `ctx.lineTo`

👉 te bajan bastante la nota, aunque “se vea bien”.

---

Si quieres, en el siguiente paso revisamos **tu archivo completo** y lo dejamos listo como si fuera entrega final (con comentarios bien puestos y estilo que el profe espera).

las indiaciones que me diste si tienen el código base que dió mi profesor?

Sí — y esto es importante decirlo bien claro — **no me inventé una estructura nueva**, trabajé directamente sobre lo que te dio tu profesor.

Pero ojo: **lo extendí**, porque el código base está incompleto a propósito (es una guía, no la solución).

Te explico exactamente cómo encaja todo con SU base:

---

## 🔷 Lo que respeté tal cual del profesor

Tu profe te dio:

- `drawPixel(ctx, x, y, color)` ✅ (lo usé sin cambiar lógica)
- Firma de `bresenhamLine(...)` ✅ (la llenamos)
- Firma de `getPolygonVertices(...)` ✅ (la llenamos)
- Enfoque modular ✅ (todo separado en funciones)

Eso se mantiene.

---

## 🔷 Lo que agregué (y es obligatorio agregar)

Tu profe **no te dio todo** porque esto hace parte del parcial. Yo añadí:

### 1. ✅ Implementación real de Bresenham

Porque él solo dejó el comentario:

JavaScript

---

### 2. ✅ Función para dibujar el polígono

Esto **no venía**, pero es necesario para usar los vértices:

JavaScript

---

### 3. ✅ Algoritmo de circunferencia (Punto Medio)

Esto tampoco venía, pero el enunciado lo exige:

> "circunferencias… todo píxel a píxel"

---

### 4. ✅ `main()` y `window.onload`

Esto es clave: el enunciado dice:

> "al cargar la página"

Tu profe no te lo dio explícito, pero **es obligatorio implementarlo**.

---

## 🔷 Algo que debes corregir (detalle importante)

Tu función base:

JavaScript

Pero en Bresenham yo usé:

JavaScript

👉 Eso implica que **ctx debe ser global o accesible**, por eso puse:

JavaScript

Si no haces eso, te va a fallar.

---

## 🔷 Conclusión (sin adornos)

Sí estás siguiendo el código base, pero:

- ❌ Si solo usas lo que dio el profe → no completas el ejercicio
- ✅ Si lo extiendes como te mostré → cumples todo el enunciado

Tu profesor espera precisamente eso:
que tomes esa base y la conviertas en una solución completa.

---

Si quieres hacerlo aún más "profe-level", en el siguiente paso puedo:

- ↳ revisar tu archivo final completo
- ↳ corregirte comentarios técnicos (eso da puntos)
- y ↳ dejarte el código listo para entrega + commits perfectos

Solo pégame lo que llevas.

así voy, crees que está bien?
JavaScript

```
// Estructura base dada por el profesor
/**
 * Universidad - Facultad de Ingeniería
 * Asignatura: Introducción a la Computación Gráfica
 * * Estudiante: Klaudia Sophia Pérez Riaño
 * * Tarea: Implementar los algoritmos de rasterización manual.
 */

// Función de apoyo para dibujar un píxel individual
function drawPixel(ctx, x, y, color = "#000000") {
    ctx.fillStyle = color;
    ctx.fillRect(Math.floor(x), Math.floor(y), 1, 1);
}
```

```
/**
 * Implementación del algoritmo de Bresenham para
líneas.
 * @param {number} x0, y0 - Coordenadas iniciales
 * @param {number} x1, y1 - Coordenadas finales
 * @returns {void}
 */
function bresenhamLine(x0, y0, x1, y1, color) {
    // Desarrollo del estudiante
    let dx= Math.abs(x1 - x0);
let dy= Math.abs(y1 - y0);

// dirección del movimiento
let sx= (x0 < x1) ? 1 : -1;
let sy= (y0 < y1) ? 1 : -1;

// parámetro de decisión inicial
let err= dx - dy;

while (true) {
    //dibujar pixel actual
    drawPixel(ctx, x0, y0, color);
    //condición de parada
    if (x0=== x1 && y0===y1)
        break;
    let e2 =2*err;
      //Si e2>-dy:se ajusta el error y se avanza en X
      if (e2>-dy){
        err -=dy;
        x0 +=sx;
      }
      //Si e2<dx:se ajusta el error y se avanza en Y
         if (e2 < dx) {
        err +=dx;
        y0 +=sy;
    }

}
}
//función circunferencia con punto medio
```

```
function midpointCircle(cx, cy, r, color) {
    let x = 0;
    let y = r
    //parámetro de desición inicial
    let p=1-r;
    //función interna para aprovechar la simetría
      function drawCirclePoints(cx, cy, x, y) {
        drawPixel(ctx, cx + x, cy + y, color);
        drawPixel(ctx, cx - x, cy + y, color);
        drawPixel(ctx, cx + x, cy - y, color);
        drawPixel(ctx, cx - x, cy - y, color);
        drawPixel(ctx, cx + y, cy + x, color);
        drawPixel(ctx, cx - y, cy + x, color);
        drawPixel(ctx, cx + y, cy - x, color);
        drawPixel(ctx, cx - y, cy - x, color);
    }
        while(x<=y){
        drawCirclePoints(cx, cy, x, y);
    //Si p<0:el punto medio está dentro del círculo
        if (p < 0) {
            p = p + 2 * x + 3;
        } else {
    //Si p >= 0: el punto medio está fuera del círculo
            p = p + 2 * (x - y) + 5;
            y--;
        }
        x++;
    }

}



/**
 * Calcula los vértices de un polígono regular.
 * @param {number} centerX, centerY - Centro
 * @param {number} sides - Número de lados
 * @param {number} radius - Radio
 * @returns {Array} Arreglo de objetos {x, y}
 */
function getPolygonVertices(centerX, centerY, sides, radius)
```

```
{
    // Desarrollo del estudiante (Uso de Math.sin/Math.cos
y retorno de datos)
}
```

Vas bien, pero no está listo para entregar. Te marco lo que está sólido y lo que te puede bajar puntos.

---

## 🔷 Lo que tienes bien (esto sí suma)

- ✅ **Bresenham completo**: cubres todos los octantes (esto está bien hecho)
- ✅ Uso correcto de `dx`, `dy`, `sx`, `sy`
- ✅ **Midpoint Circle**: la lógica está correcta
- ✅ Uso de simetría en 8 octantes
- ✅ Comentarios básicos (vas por buen camino)

---

## 🔻 Problemas que sí debes corregir

### 1. ❌ `ctx` no está definido

Estás usando:

JavaScript

pero **nunca declaraste** `ctx`.

👉 Esto en ejecución rompe todo.

**Solución mínima:**
más adelante debes tener algo como:

JavaScript

o hacerlo global:

JavaScript

---

## 2. ❌ Falta TODA la parte del polígono

Tu función:

```
function getPolygonVertices(...) {
```

JavaScript

👉 Así pierdes varios criterios:

- estructuras de datos ❌
- trigonometría ❌
- integración ❌

---

## 3. ❌ No estás dibujando nada realmente (falta integración)

Tienes algoritmos sueltos, pero el enunciado pide:

> "al cargar la página generar la figura"

👉 Te falta:

- generar `n` aleatorio
- calcular vértices
- dibujar el polígono
- dibujar circunferencias en cada vértice

---

## 4. ⚠️ Comentarios técnicos: están muy básicos

Tu profe pidió esto explícitamente:

> explicar la lógica del parámetro de decisión

Ejemplo tuyo:

JavaScript

👉 Eso es descripción, no explicación.

Debería ser algo más así:

```
// err representa la diferencia acumulada entre la línea ideal y el píxel rasteri
```

JavaScript

Si no mejoras esto, te pueden bajar por “documentación pobre”.

---

### 5. ⚠️ Detalle en midpoint (no está mal, pero ojo)

Usaste:

JavaScript

y

JavaScript

✅ Eso es correcto (forma equivalente)

Pero asegúrate de poder **explicarlo si te preguntan**, porque no es la versión más común.

---

## 🔷 Lo que te falta para que quede completo

Te faltan 3 piezas clave:

### 1. Función de vértices (obligatoria)

```javascript
function getPolygonVertices(centerX, centerY, sides, radius) {
    let vertices = [];
    let angleStep = (2 * Math.PI) / sides;

    for (let i = 0; i < sides; i++) {
        let angle = i * angleStep;

        let x = centerX + radius * Math.cos(angle);
        let y = centerY + radius * Math.sin(angle);

        vertices.push({ x, y });
    }
```

JavaScript

---

## 2. Dibujar el polígono

```javascript
function drawPolygon(vertices, color) {
    for (let i = 0; i < vertices.length; i++) {
        let current = vertices[i];
        let next = vertices[(i + 1) % vertices.length];

        bresenhamLine(current.x, current.y, next.x, next.y, color);
```

JavaScript

---

## 3. Integración final (esto es lo que realmente evalúan)

```javascript
window.onload = function () {
    let canvas = document.getElementById("canvas");
    window.ctx = canvas.getContext("2d");

    let sides = Math.floor(Math.random() * 6) + 5;
    let radius = 100;
```

```javascript
    let centerX = canvas.width / 2;
    let centerY = canvas.height / 2;

    let vertices = getPolygonVertices(centerX, centerY, sides, radius);

    drawPolygon(vertices, "#000000");

    vertices.forEach(v => {
        midpointCircle(v.x, v.y, radius / 4, "#000000");
```

JavaScript

---

## 🔷 Veredicto realista

Si entregas como vas:

👉 Apruebas la idea, pero **no cumples el enunciado completo**
👉 Probable nota: media (te faltan partes grandes)

Si agregas lo que te falta:

👉 Cumples todo: algoritmos + integración + estructura
👉 Ya compites por nota alta

---

Si quieres, en el siguiente paso te hago algo clave para subir nota fácil:
**te corrijo exactamente los comentarios técnicos para que parezcan de matrícula de honor** (eso suele marcar diferencia).

listo, para la función de getPolygonVertices, crees que estaría bien realizar la implementación con fuciones trigonometrícas, y si sí, de qué manera lo puedo hacer?

Sí, no solo está bien: **es la forma correcta**. Si no usas trigonometría aquí, básicamente estarías improvisando y se nota.

Te explico el porqué (que es lo que tu profe quiere ver que entiendes):

---

## 🔷 Idea clave

Un polígono regular se puede ver como puntos distribuidos uniformemente sobre una circunferencia.

Entonces:

- Cada vértice está a una distancia `radius` del centro
- Y cada uno está separado por un ángulo constante

Ese ángulo es:

JavaScript

---

## 🔷 Cómo se calcula cada vértice

Para cada vértice:

```
x = centroX + radio * cos(ángulo)
y = centroY + radio * sin(ángulo)
```

Eso no es opcional: **es geometría básica del plano**.

---

## 🔷 Implementación correcta (y bien justificada)

Te dejo una versión que además tiene comentarios que sí suman puntos:

```javascript
function getPolygonVertices(centerX, centerY, sides, radius) {
    let vertices = [];

    // Incremento angular entre vértices consecutivos
    let angleStep = (2 * Math.PI) / sides;

    for (let i = 0; i < sides; i++) {
```

```
        // Ángulo actual respecto al eje X positivo
        let angle = i * angleStep;

        // Conversión de coordenadas polares a cartesianas
        let x = centerX + radius * Math.cos(angle);
        let y = centerY + radius * Math.sin(angle);

        vertices.push({ x, y });
    }
```

JavaScript

---

## 🔷 Detalle que te puede subir puntos (si lo entiendes)

En canvas:

- El eje **Y crece hacia abajo**, no hacia arriba como en matemáticas

👉 Esto NO rompe el algoritmo, pero:

- El polígono queda “rotado” o invertido visualmente

Si quisieras controlarlo, podrías hacer:

JavaScript

Eso hace que:

- el primer vértice quede arriba (más “bonito” visualmente)

---

## 🔷 Error común (evítalo)

No hagas esto:

JavaScript

👉 Eso está mal porque:

- no estás usando radianes

- no distribuyes uniformemente

---

## 🔷 Conclusión clara

- ✅ Sí, trigonometría es obligatoria aquí
- ✅ `cos` y `sin` son la forma correcta
- ✅ Estás cumpliendo exactamente lo que el profe espera (uso matemático + estructura de datos)

---

Si quieres, el siguiente paso inteligente es:
te reviso **cómo integras esto con el polígono**, porque ahí es donde muchos se equivocan conectando vértices.

esto no podría ser mejor de esta manera?
vertices.push({ x: x, y: y });

Sí, **es totalmente válido**, pero no es "mejor" en ningún sentido real.

En JavaScript tienes esta abreviación:

```javascript
```

y esta forma explícita:

```javascript
```

👉 Ambas hacen exactamente lo mismo.

---

## 🔷 Entonces, ¿cuál usar?

- ✅ `{ x, y }` → más limpio, más moderno, el estándar hoy en día
- ✅ `{ x: x, y: y }` → más explícito, útil si estás empezando o quieres claridad total

---

## 🔷 Para tu caso (parcial)

Aquí hay que ser estratégico, no solo "correcto":

- Tu profesor está evaluando **claridad y estilo**
- No estás en producción, estás en evaluación

👉 Yo usaría:

JavaScript

porque:

- se ve más explícito
- evita cualquier ambigüedad al revisar
- demuestra intención clara (eso suma en revisión manual)

---

## 🔷 Conclusión realista

- ❌ No es mejor técnicamente
- ✅ Puede ser mejor **académicamente** dependiendo del profe

---

Si quieres subir nivel de verdad, preocúpate más por esto:
👉 que puedas explicar **por qué estás usando cos y sin**
👉 y cómo se distribuyen los ángulos

Eso vale mucho más que la sintaxis del objeto.

listo, ahora según lo que entiendo debo crear la funcion prinicpal, para unir las anteriores y que ahora si se dibuje

todo completo

Sí: esa es la pieza que te falta, y es donde realmente se demuestra si entendiste el flujo o solo copiaste funciones.

Pero cuidado con una idea equivocada: **la función principal no "une por magia" lo anterior**, simplemente orquesta el orden correcto de ejecución. Si el orden está mal, todo se dibuja mal aunque los algoritmos estén perfectos.

---

## 🔷 Qué debe hacer la función principal

1. Obtener el canvas y el contexto
2. Definir centro y radio
3. Generar `n` aleatorio (5 a 10)
4. Calcular vértices del polígono
5. Dibujar el polígono (Bresenham)
6. Dibujar circunferencias en cada vértice (Punto Medio)
7. Ejecutarse al cargar la página

---

## 🔷 Implementación correcta (estructura final)

```
function main() {
    // 1. Canvas y contexto
    const canvas = document.getElementById("canvas");
    window.ctx = canvas.getContext("2d");

    // 2. Parámetros base
    let centerX = canvas.width / 2;
    let centerY = canvas.height / 2;

    let radius = 100;

    // 3. Número aleatorio de lados (5 a 10)
    let sides = Math.floor(Math.random() * 6) + 5;

    // 4. Generar vértices del polígono
    let vertices = getPolygonVertices(centerX, centerY, sides, radius);
```

```javascript
    // 5. Dibujar polígono (conecta vértices)
    drawPolygon(vertices, "#000000");

    // 6. Dibujar circunferencias en cada vértice
    for (let i = 0; i < vertices.length; i++) {
        midpointCircle(vertices[i].x, vertices[i].y, radius / 4, "#000000");
    }
}
```

JavaScript

---

## 🔷 Lo que tu profe está evaluando aquí (y muchos fallan)

### ✅ Orden lógico

Si dibujas antes de generar vértices → error conceptual.

### ✅ Modularidad

No estás metiendo todo dentro de `main`, solo coordinas.

### ✅ Integración real

Aquí se ve si Bresenham + punto medio + trigonometría realmente funcionan juntos.

---

## 🔷 Error común (evítalo)

No hagas esto:

```javascript
window.onload = function () {
```

JavaScript

👉 Eso te baja puntos en “diseño y modularidad”.

---

## 🔷 Veredicto realista

Si agregas esta función:

✅ ya tienes el proyecto completo funcional
✅ cumples el enunciado al 100%
✅ estás en nivel de entrega sólida (no parcial incompleto)

---

Si quieres el siguiente paso útil de verdad, puedo ayudarte a:

- ↳ revisar si tu canvas está bien configurado en HTML
- o ↳ hacerte una versión "lista para entregar con commits exactos" (esto suele subir nota bastante)

Solo dime.

function drawScene() { let centerX = canvas.width / 2; let centerY = canvas.height / 2; //número aleatorio de lados entre 5 y 10 let sides = Math.floor(Math.random() * 6) + 5; let R = 150; let vertices = getPolygonVertices(centerX, centerY, sides, R); //dibujar el polígono for (let i = 0; i < sides; i++) { let v1 = vertices[i]; let v2 = vertices[(i + 1) % sides]; //conecta último con primero voy así, pero yo sé que todavía me falta la parte de los for

Vas bien encaminada, pero ahora mismo estás en el punto donde muchos se confunden: tienes la idea, pero te falta cerrar la lógica de ejecución y evitar pequeños errores que luego rompen todo.

Te marco lo importante sin rodeos:

---

## 🔷 1. Te falta lo más básico dentro de esa función

En tu `drawScene()` no aparece esto:

- obtener `canvas`
- obtener `ctx`

- limpiar o inicializar el canvas (opcional pero recomendable)
- realmente dibujar las líneas con Bresenham

Ahora mismo solo estás calculando vértices y recorriéndolos, pero **no estás dibujando nada todavía**.

---

## 🔷 2. Tu loop está incompleto (te falta la llamada real)

Tienes esto:

JavaScript

Pero te falta lo esencial:

👉 llamar a Bresenham:

JavaScript

Sin eso, el polígono no existe visualmente.

---

## 🔷 3. Lo que te falta después del polígono

El enunciado no se queda en el polígono. También debes:

👉 dibujar circunferencias en cada vértice

Eso va en otro `for` (sí, necesitas dos recorridos separados o uno combinado con cuidado).

---

## 🔷 4. Versión correcta de tu función (ordenada y completa)

Así debería quedar tu `drawScene()`:

```javascript
function drawScene() {
    const canvas = document.getElementById("canvas");
    window.ctx = canvas.getContext("2d");

    let centerX = canvas.width / 2;
    let centerY = canvas.height / 2;

    // número aleatorio de lados entre 5 y 10
    let sides = Math.floor(Math.random() * 6) + 5;

    let R = 150;

    let vertices = getPolygonVertices(centerX, centerY, sides, R);

    // 🔷 1. Dibujar polígono
    for (let i = 0; i < sides; i++) {
        let v1 = vertices[i];
        let v2 = vertices[(i + 1) % sides];

        bresenhamLine(v1.x, v1.y, v2.x, v2.y, "#000000");
    }

    // 🔷 2. Dibujar circunferencias en cada vértice
    for (let i = 0; i < vertices.length; i++) {
        midpointCircle(vertices[i].x, vertices[i].y, R / 4, "#000000");
```

JavaScript

---

## 🔷 5. Error conceptual que estás evitando sin darte cuenta

Tu intuición de “me falta la parte de los for” es correcta, pero ojo con esto:

- No es “más for = mejor”
- Es “cada for tiene una responsabilidad clara”

👉 Uno es conectividad (líneas)
👉 Otro es ornamentación (círculos)

Si los mezclas sin orden, el código funciona pero **pierde estructura (y eso el profe lo mira)**.

---

## 🔷 6. Lo que sigue (importante para nota)

Si quieres asegurar nota alta, revisa esto después:

- ¿estás llamando `drawScene()` en `window.onload` ?
- ¿ctx es global o accesible en todas las funciones?
- ¿bresenham y midpoint usan el mismo ctx sin error?

---

Si quieres, en el siguiente paso puedo ayudarte a:
👉 ↪ revisar todo tu archivo como si fuera entrega final
👉 o ↪ decirte exactamente qué commits te faltan para cumplir el criterio del profesor sin perder puntos innecesarios

listo, ya entonces tengo todo el js, supongo que debo pasar al html

Sí, pero no es “solo pasar al HTML”. Aquí es donde muchos creen que ya terminó y luego el código no corre.

Tu JS puede estar perfecto y aun así no funcionar si el HTML está mal conectado.

---

## 🔷 Qué debe tener tu HTML (mínimo obligatorio)

1. Un `<canvas>` con id correcto
2. Tamaño definido (si no, no dibuja o dibuja raro)
3. Cargar tu archivo `.js` al final
4. Asegurar que el script corre después de cargar el DOM

---

## 🔷 HTML correcto (estructura base)

```html
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
```

```html
    <title>Parcial 2 - Computación Gráfica</title>

    <style>
        canvas {
            border: 1px solid black;
        }
    </style>
</head>

<body>
```

HTML

---

## 🔷 Errores típicos (y aquí se caen muchos)

### ❌ 1. ID incorrecto

Si en JS tienes:

JavaScript

pero en HTML pusiste otro nombre → no dibuja nada.

---

### ❌ 2. Script en `<head>` sin control

Si lo pones arriba sin `defer`, el canvas aún no existe.

---

### ❌ 3. No definir tamaño del canvas

Si no defines:

HTML

el dibujo puede verse estirado o invisible.

---

## 🔷 Orden correcto de ejecución (esto es lo que evalúan indirectamente)

1. HTML carga canvas
2. JS se carga
3. `window.onload` ejecuta `drawScene()` o `main()`
4. Se generan vértices
5. Se dibuja todo

---

## 🔷 Punto crítico (te lo digo directo)

Si algo falla ahora, no va a ser por algoritmos. Va a ser por:

- ID mal escrito
- ctx no definido globalmente
- función principal no ejecutándose
- script mal cargado

---

## 🔷 Lo siguiente que deberías hacer

Antes de decir "ya está listo", verifica:

- Abres consola del navegador (F12)
- No hay errores rojos
- Ves el canvas con borde
- Se dibuja algo al refrescar

---

Si quieres, el siguiente paso puede ser útil de verdad:
👉 te reviso tu estructura completa como entrega final (JS + HTML + lógica de commits) y te digo si estás en nivel 3, 4 o 5 de nota realista según cómo lo evalúan normalmente.

al ejecutar el código únicamente se dibuja el canvas

Eso ya es una señal clara: **tu JS no se está ejecutando o se está rompiendo antes de dibujar algo**. No es un problema de Bresenham ni de trigonometría; es de flujo básico.

Vamos por partes, sin suposiciones:

---

## 🔴 1. Lo más probable: no estás llamando la función principal

Tú tienes algo como `drawScene()`, pero si no la ejecutas, no pasa nada.

Verifica que tengas **una de estas dos cosas**:

JavaScript

o

JavaScript

Si eso no existe → el canvas aparece, pero está vacío.

---

## 🔴 2. Segundo problema típico: `ctx` no está bien definido

Dentro de tus funciones estás usando:

JavaScript

Si `ctx` no existe globalmente, el código se detiene en silencio o lanza error.

Debes tener esto dentro de `drawScene()`:

JavaScript

👉 Si no usas `window.ctx`, las otras funciones no lo ven.

## 🔴 3. Revisa esto en consola (MUY importante)

Abre F12 → Console

Busca:

- ❌ `ctx is not defined`
- ❌ `bresenhamLine is not defined`
- ❌ `getPolygonVertices is not defined`

Si hay uno de esos, el problema no es lógico, es de carga o alcance.

---

## 🔴 4. Otro error frecuente: orden del script

Tu HTML debe tener esto:

HTML

```
```

👉 al final del body

Si lo pusiste en `<head>` sin `defer`, el canvas todavía no existe y falla todo.

---

## 🔴 5. Diagnóstico rápido (haz esto ahora)

Agrega esto al inicio de `drawScene()`:

JavaScript

```
```

Si NO aparece en consola → la función nunca se está llamando.

---

## 🔴 6. Causa más probable por lo que describes

Por experiencia con este tipo de parciales, lo más común es:

👉 **no estás llamando `drawScene()` o `window.onload` está mal**

## ✅ Resumen realista

Si solo ves el canvas:

- JS no está ejecutándose ❌
- o se está rompiendo antes del primer drawPixel ❌
- o no hay llamada inicial ❌

---

## 👉 Qué necesito para ayudarte a cerrarlo en 1 paso

Pásame tu:

- HTML completo
- y cómo estás llamando `drawScene()`

y te digo exactamente dónde está el fallo sin adivinar.

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Parcial 2 - Computación Gráfica</title>
  <style>
    canvas {
      border: 1px solid black;
    }
  </style>
</head>
<body>
  <canvas id="canvas" width="600" height="600">
</canvas>
  <script src="index.js"></script>
</body>
</html>
```

JavaScript

```
// Estructura base dada por el profesor
/**
 * Universidad - Facultad de Ingeniería
 * Asignatura: Introducción a la Computación Gráfica
 * * Estudiante: Klaudia Sophia Pérez Riaño
 * * Tarea: Implementar los algoritmos de rasterización
manual.
 */

// Función de apoyo para dibujar un píxel individual
function drawPixel(ctx, x, y, color = "#000000") {
    ctx.fillStyle = color;
    ctx.fillRect(Math.floor(x), Math.floor(y), 1, 1);
}

/**
 * Implementación del algoritmo de Bresenham para
líneas.
 * @param {number} x0, y0 - Coordenadas iniciales
 * @param {number} x1, y1 - Coordenadas finales
 * @returns {void}
 */
function bresenhamLine(x0, y0, x1, y1, color) {
    // Desarrollo del estudiante
    let dx= Math.abs(x1 - x0);
let dy= Math.abs(y1 - y0);

// dirección del movimiento
let sx= (x0 < x1) ? 1 : -1;
let sy= (y0 < y1) ? 1 : -1;

// parámetro de decisión inicial
let err= dx - dy;

while (true) {
    //dibujar pixel actual
    drawPixel(ctx, x0, y0, color);
    //condición de parada
    if (x0=== x1 && y0===y1)
```

```
        break;
    let e2 =2*err;
      //Si e2>-dy:se ajusta el error y se avanza en X
      if (e2>-dy){
        err -=dy;
        x0 +=sx;
      }
      //Si e2<dx:se ajusta el error y se avanza en Y
         if (e2 < dx) {
        err +=dx;
        y0 +=sy;
    }

}
}
//función circunferencia con punto medio
function midpointCircle(cx, cy, r, color) {
    let x = 0;
    let y = r
    //parámetro de desición inicial
    let p=1-r;
    //función interna para aprovechar la simetría
       function drawCirclePoints(cx, cy, x, y) {
        drawPixel(ctx, cx + x, cy + y, color);
        drawPixel(ctx, cx - x, cy + y, color);
        drawPixel(ctx, cx + x, cy - y, color);
        drawPixel(ctx, cx - x, cy - y, color);
        drawPixel(ctx, cx + y, cy + x, color);
        drawPixel(ctx, cx - y, cy + x, color);
        drawPixel(ctx, cx + y, cy - x, color);
        drawPixel(ctx, cx - y, cy - x, color);
    }
        while(x<=y){
        drawCirclePoints(cx, cy, x, y);
    //Si p<0:el punto medio está dentro del círculo
        if (p < 0) {
            p = p + 2 * x + 3;
        } else {
    //Si p >= 0: el punto medio está fuera del círculo
            p = p + 2 * (x - y) + 5;
```

```
            y--;
        }
        x++;
    }

}

/**
 * Calcula los vértices de un polígono regular.
 * @param {number} centerX, centerY - Centro
 * @param {number} sides - Número de lados
 * @param {number} radius - Radio
 * @returns {Array} Arreglo de objetos {x, y}
 */
function getPolygonVertices(centerX, centerY, sides, radius)
{
    // Desarrollo del estudiante (Uso de Math.sin/Math.cos
y retorno de datos)
    let vertices = [];
    //ángulo entre cada vértice
    let angleStep = (2 * Math.PI) / sides;
    for (let i = 0; i < sides; i++) {
    //ángulo respecto al eje X positivo
    let angle = i * angleStep;
    //cálculo de coordenadas usando trigonometría
    let x = centerX + radius * Math.cos(angle);
    let y = centerY + radius * Math.sin(angle);
    vertices.push({ x: x, y: y });
}
return vertices;
}

//función principal, que arma y dibuja una escena
completa en el canvas
function drawScene() {
    let centerX = canvas.width / 2;
    let centerY = canvas.height / 2;
    //número aleatorio de lados entre 5 y 10
    let sides = Math.floor(Math.random() * 6) + 5;
    let R = 150;
```

```
    let vertices = getPolygonVertices(centerX, centerY, sides,
R);
    //dibujar el polígono
    for (let i = 0; i < sides; i++) {
        let v1 = vertices[i];
        let v2 = vertices[(i + 1) % sides]; //conecta último con
primero
        //llamar a bresenhamLine
            bresenhamLine(
            Math.round(v1.x),
            Math.round(v1.y),
            Math.round(v2.x),
            Math.round(v2.y),
            "black"
        );
    }
    //dibujar circunferencias en cada vértice
    for (let v of vertices) {
        midpointCircle(
            Math.round(v.x),
            Math.round(v.y),
            R / 4,
            "red"
        );
    }
}
```

Ya encontré el problema. No es pequeño: es **la razón completa por la que no se dibuja nada**.

---

## 🔴 ERROR PRINCIPAL (el que está rompiendo todo)

Tú definiste la función:

```
</> JavaScript
```

pero **nunca la ejecutas**.

👉 Resultado: el canvas se crea, pero jamás dibujas nada.

---

## ✅ SOLUCIÓN INMEDIATA

Al final de tu `index.js`, agrega esto:

JavaScript

o si quieres más explícito:

JavaScript

---

## 🔴 SEGUNDO ERROR CRÍTICO (este también te está frenando)

Dentro de `drawScene()` usas:

JavaScript

👉 Pero **canvas NO existe en JS** porque nunca lo declaraste.

---

## ✅ SOLUCIÓN

Debes agregar esto al inicio de `drawScene()`:

JavaScript

---

## 🔴 TERCER PROBLEMA (te puede romper todo silenciosamente)

En `bresenhamLine` usas `ctx`, pero solo funciona si `window.ctx` existe.

👉 Por eso es clave el paso anterior.

---

## ✅ TU FUNCIÓN PRINCIPAL CORREGIDA

Así debería quedar:

```javascript
function drawScene() {
    const canvas = document.getElementById("canvas");
    window.ctx = canvas.getContext("2d");

    let centerX = canvas.width / 2;
    let centerY = canvas.height / 2;

    let sides = Math.floor(Math.random() * 6) + 5;
    let R = 150;

    let vertices = getPolygonVertices(centerX, centerY, sides, R);

    // polígono
    for (let i = 0; i < sides; i++) {
        let v1 = vertices[i];
        let v2 = vertices[(i + 1) % sides];

        bresenhamLine(
            Math.round(v1.x),
            Math.round(v1.y),
            Math.round(v2.x),
            Math.round(v2.y),
            "black"
        );
    }

    // círculos
    for (let v of vertices) {
        midpointCircle(
            Math.round(v.x),
            Math.round(v.y),
            R / 4,
            "red"
        );
```

JavaScript

```
    }
}
```

---

## 🔴 Y ESTO ES LO QUE TE FALTABA EN EL JS

Al final del archivo:

JavaScript

---

## 🔷 RESUMEN REALISTA

Tu código estaba bien en algoritmos.

El problema era:

- ❌ nunca llamaste la función principal
- ❌ no inicializaste canvas dentro de JS
- ❌ dependías de variables que no existen globalmente

---

## ✅ VEREDICTO

Con estos 2 cambios:

- `canvas + ctx dentro de drawScene`
- `window.onload = drawScene`

👉 ya debería dibujar todo correctamente.

---

Si quieres, el siguiente paso puedo ayudarte a algo que sí te sube nota:
👉 ↳ cómo justificar cada algoritmo en comentarios como “respuesta de parcial” (eso es lo que diferencia un 3.5 de un 4.5 en este tipo de trabajos).

↳ window.onload \= drawScene;

esto va al final de todo el código o dentro de una función?

Va **al final del archivo JS, pero NO dentro de ninguna función**.

---

## 🔷 Por qué

Esto:

```
JavaScript
```

es una instrucción de configuración global del navegador. Lo que hace es:

- "cuando termine de cargar toda la página, ejecuta `drawScene()` "

No pertenece a una función, ni debe ir dentro de otra, porque si lo metes ahí, puede que:

- no se registre correctamente
- o se ejecute demasiado tarde / nunca

---

## 🔷 Dónde exactamente ponerlo

Al final de tu `index.js` , después de todas las funciones:

```
// ... todas tus funciones (bresenham, midpoint, etc.)

function drawScene() {
    // ...
}

JavaScript
```

---

## 🔷 Alternativa equivalente (por si la ves en clase)

También es válido:

JavaScript

---

## 🔷 Idea clave (importante para el parcial)

- `drawScene()` = lo que tú defines
- `window.onload` = cuándo se ejecuta

Si no lo conectas ahí, tu código es correcto… pero **no se ejecuta nunca**.

---

Si después de esto ya te dibuja, el siguiente paso lógico es revisar si todo se ve centrado y limpio (porque a veces sí dibuja, pero "mal ubicado" o con errores visuales pequeños).

listo, me gustaría que el canvas quede en el centro y en la parte de arriba aparezca el título de la página, pero únicamente indicame cmo hacerlo, no me des el código

Perfecto, eso ya es tema de HTML + CSS básico, no de algoritmos.

---

## 🔷 Para centrar el canvas

Tienes dos opciones típicas:

### ✅ Opción recomendada (CSS con flexbox)

La idea es que el `<body>` se convierta en un contenedor flexible que:

- alinee horizontalmente al centro
- alinee verticalmente al centro o con margen superior si prefieres

Lo que debes hacer es:

- poner el `body` como contenedor flex
- activar alineación centrada en ambos ejes
- darle altura completa a la pantalla ( `100vh` )
- y quitar márgenes por defecto del navegador

---

### ✅ Alternativa simple

Puedes:

- envolver el canvas en un `<div>`
- centrar ese `div` con `margin auto`
- o usar `text-align: center` si solo quieres centrado horizontal

---

### 🔷 Para el título arriba

Esto es HTML puro:

- agregas un elemento de encabezado (tipo título principal)
- lo colocas antes del canvas dentro del `<body>`
- normalmente se usa un `<h1>`

---

### 🔷 Para que se vea “bien presentado”

Si quieres que quede estilo entrega formal:

- título arriba centrado
- canvas debajo con separación (margen superior)
- fondo limpio o blanco
- borde del canvas visible (ya lo tienes)

---

### 🔷 Lo importante que debes entender

- HTML = estructura (título + canvas)
- CSS = posición (centrar, espacios)
- JS = dibujo (lo que ya hiciste)

---

Si quieres, después te digo cómo dejarlo con apariencia “de parcial bien presentado” (que suele sumar puntos aunque no lo pidan explícitamente).

revisa que tenga todas las condiciones que se pidieron
//JavaScript
// Estructura base dada por el profesor

```
/**
 * Universidad - Facultad de Ingeniería
 * Asignatura: Introducción a la Computación Gráfica
 *
 * Estudiante: Klaudia Sophia Pérez Riaño
 *
 * Tarea: Implementar los algoritmos de rasterización manual.
 */
// Función de apoyo para dibujar un píxel individual
function drawPixel(ctx, x, y, color = "#0a472e") {
    ctx.fillStyle = color;
    ctx.fillRect(Math.floor(x), Math.floor(y), 1, 1);
}

/**
 * Implementación del algoritmo de Bresenham para líneas.
 * @param {number} x0, y0 - Coordenadas iniciales
 * @param {number} x1, y1 - Coordenadas finales
 * @returns {void}
 */
function bresenhamLine(ctx, x0, y0, x1, y1, color) {
    let dx = Math.abs(x1 - x0);
    let dy = Math.abs(y1 - y0);
    // dirección del movimiento
    let sx = (x0 < x1) ? 1 : -1;
    let sy = (y0 < y1) ? 1 : -1;
    // parámetro de decisión inicial
    // err representa el error acumulado entre la línea ideal y la rasterizada
```

```
    let err = dx - dy;
    while (true) {
        // dibujar pixel actual
        drawPixel(ctx, x0, y0, color);
        // condición de parada
        if (x0 === x1 && y0 === y1) break;
        let e2 = 2 * err;
        // Si e2 > -dy: se ajusta el error y se avanza en X
        if (e2 > -dy) {
            err -= dy;
            x0 += sx;
        }
        // Si e2 < dx: se ajusta el error y se avanza en Y
        if (e2 < dx) {
            err += dx;
            y0 += sy;
        }
    }
}
// función circunferencia con punto medio
function midpointCircle(ctx, cx, cy, r, color) {
    let x = 0;
    let y = r;
    // parámetro de decisión inicial
    // p determina si el punto medio está dentro o fuera del círculo
    let p = 1 - r;
    // función interna para aprovechar la simetría de los 8 octantes
    function drawCirclePoints(cx, cy, x, y) {
        drawPixel(ctx, cx + x, cy + y, color);
        drawPixel(ctx, cx - x, cy + y, color);
        drawPixel(ctx, cx + x, cy - y, color);
        drawPixel(ctx, cx - x, cy - y, color);
        drawPixel(ctx, cx + y, cy + x, color);
        drawPixel(ctx, cx - y, cy + x, color);
        drawPixel(ctx, cx + y, cy - x, color);
        drawPixel(ctx, cx - y, cy - x, color);
    }
    while (x <= y) {
```

```
        drawCirclePoints(cx, cy, x, y);

        // Si p < 0: el punto medio está dentro del círculo
        if (p < 0) {
            p = p + 2 * x + 3;
        } else {
            // Si p >= 0: el punto medio está fuera del círculo
            p = p + 2 * (x - y) + 5;
            y--;
        }
        x++;
    }
}
/**
 * Calcula los vértices de un polígono regular.
 * @param {number} centerX, centerY - Centro
 * @param {number} sides - Número de lados
 * @param {number} radius - Radio
 * @returns {Array} Arreglo de objetos {x, y}
 */
function getPolygonVertices(centerX, centerY, sides, radius)
{
    let vertices = [];
    // ángulo entre cada vértice
    let angleStep = (2 * Math.PI) / sides;
    for (let i = 0; i < sides; i++) {
        // ángulo respecto al eje X positivo
        let angle = i * angleStep;
        // cálculo de coordenadas usando trigonometría
        let x = centerX + radius * Math.cos(angle);
        let y = centerY + radius * Math.sin(angle);
        vertices.push({
            x: Math.round(x),
            y: Math.round(y)
        });
    }
    return vertices;
}
// función principal, que arma y dibuja una escena
completa en el canvas
```

```
function drawScene() {
    const canvas = document.getElementById("canvas");
    const ctx = canvas.getContext("2d");
    let centerX = canvas.width / 2;
    let centerY = canvas.height / 2;
    let sides = Math.floor(Math.random() * 6) + 5;
    let R = 150;
    let vertices = getPolygonVertices(centerX, centerY, sides,
R);
    // polígono
    for (let i = 0; i < sides; i++) {
        let v1 = vertices[i];
        let v2 = vertices[(i + 1) % sides];
        bresenhamLine(
            ctx,
            v1.x,
            v1.y,
            v2.x,
            v2.y,
            "black"
        );
    }
    // círculos en cada vértice
    for (let v of vertices) {
        midpointCircle(
            ctx,
            v.x,
            v.y,
            R / 4,
            "red"
        );
    }
}
window.onload = drawScene;
```

Lo revisé con lupa contra lo que te pidió el enunciado y los criterios. Te digo sin suavizar: **está bastante sólido, pero todavía no está perfecto para asegurar nota alta sin observaciones del profe.**

# 🔍 Revisión contra requisitos del parcial

## ✅ 1. Polígono regular (n entre 5 y 10)

✔ Cumples:

JavaScript

- Correcto rango
- Correcto uso aleatorio

---

## ⚠️ 2. Bresenham manual (sin funciones nativas)

✔ Cumples:

- No usas `lineTo`, `stroke`, etc.
- Implementación correcta de error acumulado

🔶 Detalle menor:

- Estás pasando `ctx` dentro de la función (bien), pero luego también dependes de consistencia global. No es error, pero el profe podría preguntarte por qué no lo encapsulaste mejor.

---

## ⚠️ 3. Punto medio (circunferencia)

✔ Cumples:

- Simetría de 8 octantes correcta
- Parámetro de decisión correcto

🔶 Mejora importante:
Tu comentario dice:

JavaScript

✔ Bien, pero todavía es **descriptivo**, no explicativo matemáticamente profundo (eso es lo que suelen buscar para “excelente”).

## ⚠️ 4. Vértices del polígono (trigonometría + retorno de datos)

✅ Cumples:

- Usas `cos` y `sin` correctamente
- Retornas array de objetos ✅
- Redondeas coordenadas ✅

🔶 Detalle:

JavaScript

Esto es válido, pero estás **perdiendo precisión geométrica intencionalmente**. No está mal, pero si el profe es estricto, podría decirte que "alteras la construcción ideal".

---

## ⚠️ 5. Integración completa (lo más importante del ejercicio)

✅ Cumples:

- Generas figura completa
- Dibujas polígono
- Dibujas circunferencias en vértices
- Todo se ejecuta en `window.onload`

👉 Esto es lo que más pesa en la nota.

---

## ⚠️ 6. Validación de octantes en Bresenham

✅ Parcialmente cumplido:

- Sí manejas todos los casos (dx/dy + sx/sy)
- Pero no lo documentas explícitamente como "octantes"

👉 El enunciado lo pide como criterio pedagógico, no funcional.

---

## ⚠️ 7. Estructura de código y modularidad

✅ Bien:

- Funciones separadas correctamente

- Flujo lógico claro

🔶 Mejora leve:
No hay una función tipo `init()` o separación clara entre:

- inicialización
- lógica de dibujo

No es obligatorio, pero suma presentación.

---

ChatGPT Obtener Plus ×

### ⚠️ No validas el canvas antes de usarlo

Si el HTML falla o el id cambia, todo muere silenciosamente.

No es obligatorio, pero en evaluación real eso se considera “fragilidad del código”.

---

## 📊 Veredicto realista

### 🔵 Nivel actual: 8.5 / 10

### ✅ Por qué estás bien:

- Algoritmos correctos
- Integración completa
- Sin funciones nativas de dibujo
- Cumple el objetivo geométrico

### 🔴 Por qué no es 10:

- Comentarios poco “matemáticos”
- Falta validación y estructura más formal
- Detalles de presentación (no de lógica)

---

## 🧠 Lo que diría un profe estricto

> “Funciona bien, pero falta justificar mejor los parámetros de decisión y reforzar la formalidad del código.”

## 💡 Si quieres subirlo a 9.5–10

Solo te faltaría mejorar:

- comentarios del error de Bresenham (más matemáticos)
- explicación del parámetro p en círculo (más formal)
- opcional: limpiar estructura de inicialización

---

Si quieres, puedo hacerte algo más útil todavía:
👉 decirte exactamente qué escribir en los comentarios para que parezca “nivel excelente de parcial”, sin cambiar tu código.